Part 4 — Grundy's Game: Explanation and Python Code

Grundy's Game is similar to Standard Nim in that two players take turns making moves, and the player who cannot make a legal move loses. However, **the way you make a move is completely different**.

Your assignment starts with a single pile containing five objects:

```
initial_grundy_state = (5,)
```

Let's understand the rules, find all legal moves, determine the winning strategy, and then implement the three required Python functions.

1. What is Grundy's Game?

In Grundy's Game, a player must select one pile and **split it into two non-empty piles of unequal sizes**.

For example, suppose we have a pile of five objects:

```
● ● ● ● ●
```

We can split it in two legal ways:

Move 1: Split 5 into 1 and 4

```
● ● ● ● ●  →  ● ● ● ●  +  ●

(5,)  →  (4, 1)
```

Move 2: Split 5 into 2 and 3

```
● ● ● ● ●  →  ● ● ●  +  ● ●

(5,)  →  (3, 2)
```

Both moves are legal because the resulting piles are non-empty and have different sizes. Pasted markdown(1)

Which moves are illegal?

Consider these examples:

Split	Legal?	Reason
(5 \rightarrow 1+4)	Yes	Both piles are non-empty and unequal.
(5 \rightarrow 2+3)	Yes	Both piles are non-empty and unequal.
(4 \rightarrow 2+2)	No	The resulting piles have equal sizes.
(5 \rightarrow 0+5)	No	Both resulting piles must be non-empty.
(1 \rightarrow 0+1)	No	A pile of size 1 cannot be split legally.

Important difference from Standard Nim:

- In Standard Nim, you remove objects from a pile.
- In Grundy's Game, you split a pile into two unequal piles. You do not remove objects from the game.

The total number of objects stays the same after every move.

---

2. How do we represent the game state?

We represent a state as a Python tuple containing the sizes of all piles.

For example:

State	Meaning
`(5,)`	One pile of size 5
`(4, 1)`	Two piles of sizes 4 and 1
`(3, 2)`	Two piles of sizes 3 and 2
`(3, 1, 1)`	Three piles of sizes 3, 1, and 1
`(2, 2, 1)`	Three piles of sizes 2, 2, and 1

The tuple must follow two rules:

1. Pile sizes are sorted in **non-increasing order** (largest to smallest).
2. Empty piles are omitted.

For example:

```
(1, 4)  →  (4, 1)
```

These represent the same game state, so we always use `(4, 1)`.

Notice that a state can contain two piles of equal size, such as `(2, 2, 1)`. That is allowed. The restriction is that **a single pile cannot be split into two equal-sized piles**. Pasted markdown(1)

---

3. What does winning or losing mean?

Your assignment uses the following convention:

- `True` : The player whose turn it is can guarantee a win.
- `False` : The player whose turn it is will lose if both players play optimally.

Let's look at the simplest example.

Example: State `(1,)`

There is one pile containing one object.

Can we split it into two unequal, non-empty piles?

No. The only possible division would be:

[
1=0+1
]

But zero-sized piles are not allowed.

Therefore, `(1,)` has no legal moves.

The player whose turn it is loses:

```
grundy_is_winning((1,)) == False
```

This is our **base case**: if a state has no legal moves, return `False`.

The recursive rule

Suppose the current state has several legal moves.

- If at least one move leads to a losing state for the opponent, the current state is winning.
- If every legal move leads to a winning state for the opponent, the current state is losing.

In Python, the core logic is:

```
for next_state in grundy_successors(state):
    if not grundy_is_winning(next_state):
        return True

return False
```

This is the main idea behind our solver.

---

4. Understand the three required functions

Your starter code has three functions to complete.

Function 1: `grundy_successors(state)`

Purpose: Generate all distinct legal next states.

For the initial state `(5,)`, the function should return:

```
[(4, 1), (3, 2)]
```

How does it work?

1. Look at each pile.
2. Try splitting that pile into two positive numbers.
3. Reject splits where both numbers are equal.
4. Replace the original pile with the two new piles.
5. Sort the pile sizes from largest to smallest.
6. Store distinct states and return them in a consistent order.

For example, splitting 5 into 1 and 4 produces `(4, 1)`. Splitting it into 2 and 3 produces `(3, 2)`.

The function uses `@lru_cache(maxsize=None)` because the same states may be encountered repeatedly during the recursive search.

Function 2: `grundy_is_winning(state)`

Purpose: Determine whether the current player can force a win.

The logic is:

1. Generate all legal next states.
2. If there are no legal moves, return `False`.
3. Otherwise, examine each possible next state.
4. If any next state is losing for the opponent, return `True`.
5. If all next states are winning for the opponent, return `False`.

For example, consider `(4, 1)`.

Its only legal move is:

[
(4,1)\rightarrow(3,1,1)
]

The state `(3, 1, 1)` has a legal move:

[
(3,1,1)\rightarrow(2,1,1,1)
]

The resulting state `(2, 1, 1, 1)` has no legal moves because none of its piles can be split into two unequal, non-empty piles.

Therefore, `(2, 1, 1, 1)` is losing. That makes `(3, 1, 1)` winning, which makes `(4, 1)` losing.

In other words, the player starting at `(4, 1)` cannot win against an optimal opponent.

Function 3: `grundy_best_moves(state)`

Purpose: Return all moves that leave the opponent in a losing position.

For the initial state `(5,)`, it examines both legal moves:

- `(5,) → (4, 1)`

- `(5,) → (3, 2)`

It checks whether each resulting state is losing.

As we will see, `(4, 1)` is losing, whereas `(3, 2)` is winning.

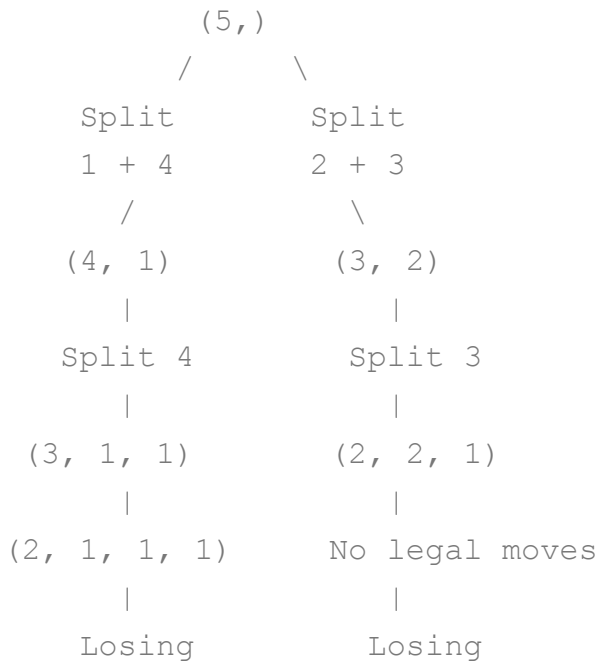
Therefore, the best move is:

```
[(4, 1)]
```

---

5. Predict the answer before coding

Let's work through the complete first level of the game tree.



Let's calculate the winning or losing status from the bottom upward.

First: `(2, 1, 1, 1)` is losing because no pile can be split legally.

Second: `(3, 1, 1)` is winning because splitting the pile of size 3 produces the losing state `(2, 1, 1, 1)`.

Third: `(4, 1)` is losing because its only move leads to `(3, 1, 1)`, which is winning for the opponent.

Fourth: `(2, 2, 1)` is losing because no pile can be split legally.

Fifth: `(3, 2)` is winning because splitting the pile of size 3 produces the losing state `(2, 2, 1)`.

Finally, `(5,)` is winning because it can move to `(4, 1)`, a losing state for the opponent.

Thus, MAX should split the pile of five into **one and four**, producing `(4, 1)`.

---

6. Complete Python Code

Copy the following code into your notebook and run it. It implements all three required functions and tests the initial state.

from functools import lru_cache

@lru_cache(maxsize=None)
def grundy_successors(state):
moves = set()

```
for i in range(len(state)):
    n = state[i]

    for a in range(1, n):
        b = n - a

        if a != b:
            next_state = list(state)
            next_state.pop(i)
            next_state.extend([a, b])

            next_state = tuple(sorted(next_state, reverse=True))
            moves.add(next_state)

return sorted(moves, reverse=True)
```

@lru_cache(maxsize=None)
def grundy_is_winning(state):
for next_state in grundy_successors(state):
if not grundy_is_winning(next_state):
return True

```
return False
```

def grundy_best_moves(state):
winning_moves = [ ]

```
for next_state in grundy_successors(state):
    if not grundy_is_winning(next_state):
        winning_moves.append(next_state)

return winning_moves
```

initial_grundy_state = (5,)

print('Initial state:', initial_grundy_state)
print('Legal first moves:', grundy_successors(initial_grundy_state))
print('Winning for player to move:', grundy_is_winning(initial_grundy_state))
print('Winning moves:', grundy_best_moves(initial_grundy_state))

Expected output

```
Initial state: (5,)
Legal first moves: [(4, 1), (3, 2)]
Winning for player to move: True
Winning moves: [(4, 1)]
```

7. Explanation of the code

Let's examine the most important parts.

A. Generating unequal splits

```
for a in range(1, n):
    b = n - a

    if a != b:
```

The variable `a` represents the size of the first new pile. The variable `b` represents the size of the second new pile.

For a pile of size 5, the possible splits are:

a	b	Legal?
1	4	Yes
2	3	Yes

a	b	Legal?
3	2	Yes, but duplicates the 2 + 3 split
4	1	Yes, but duplicates the 1 + 4 split

The `set` removes duplicate states.

B. Replacing the original pile

```
next_state = list(state)
next_state.pop(i)
next_state.extend([a, b])
```

These lines remove the selected pile from the state and replace it with the two new piles.

For example:

```
Original: (5,)
Remove:   5
Add:      4 and 1
Result:   (4, 1)
```

C. Sorting the state

```
next_state = tuple(sorted(next_state, reverse=True))
```

This ensures that the largest pile appears first.

For example:

```
(1, 3, 1) → (3, 1, 1)
```

D. Determining whether a state is winning

```
@lru_cache(maxsize=None)
def grundy_is_winning(state):
    for next_state in grundy_successors(state):
        if not grundy_is_winning(next_state):
            return True
```

```
    return False
```

If any move leads to a losing state for the opponent, the function returns `True`.

If no such move exists, the function returns `False`.

There is no explicit `if not grundy_successors(state)` check. That is intentional: if the state has no legal moves, the loop runs zero times, and the function reaches `return False` automatically.

E. Finding the winning moves

```
def grundy_best_moves(state):
    winning_moves = []

    for next_state in grundy_successors(state):
        if not grundy_is_winning(next_state):
            winning_moves.append(next_state)

    return winning_moves
```

This function returns every legal next state that is losing for the opponent.

For `(5,)`, it returns only `(4, 1)`.

---

8. Final assignment table

Your uploaded assignment also asks you to analyse several states. The results are:

State	Legal successor states	Winning or losing for player to move?
`(5,)`	`(4, 1)`, `(3, 2)`	Winning
`(1, 4)`	`(3, 1, 1)`	Losing
`(2, 3)`	`(2, 2, 1)`	Winning
`(1, 1, 3)`	`(2, 1, 1, 1)`	Winning
`(1, 2, 2)`	None	Losing

The states in the first column are shown in the assignment's notation. The code sorts them into non-increasing order before processing them.

9. Explanation to write in your assignment

Grundy's Game is a two-player game in which a player selects one pile and splits it into two non-empty piles of unequal sizes. Unlike Standard Nim, objects are not removed from a pile; the selected pile is divided into two smaller piles. A player who cannot make a legal move loses.

The grundy_successors() function generates all distinct legal successor states, sorts the pile sizes in non-increasing order, and removes duplicates. The grundy_is_winning() function uses recursion to determine whether the current player has a winning strategy. A state is winning if at least one legal move leaves the opponent in a losing state. The grundy_best_moves() function returns all such moves. Memoization avoids recalculating previously evaluated states.

For the initial state (5,), there are two legal moves: splitting the pile into 1 and 4, producing (4, 1), or splitting it into 2 and 3, producing (3, 2). The state (4, 1) is losing because its only legal move leads to (3, 1, 1), which is winning for the opponent. In contrast, (3, 2) is winning because splitting the pile of size 3 produces (2, 2, 1), a losing state.

Therefore, the initial state (5,) is winning, and the optimal first move is to split the pile into 1 and 4, producing (4, 1).

10. Key points to remember

- **Legal move:** Split one pile into two unequal, non-empty piles.
- **Illegal move:** Split a pile into two equal piles, such as (4\rightarrow2+2).
- **Terminal state:** A state with no legal splits.
- **Winning state:** At least one move leads to a losing state for the opponent.
- **Losing state:** Every legal move leads to a winning state for the opponent.
- **Memoization:** `@lru_cache(maxsize=None)` stores previous results to avoid repeated calculations.

Final answer for `(5,)`: MAX has a winning strategy. The winning first move is `(5,) → (4, 1)`.